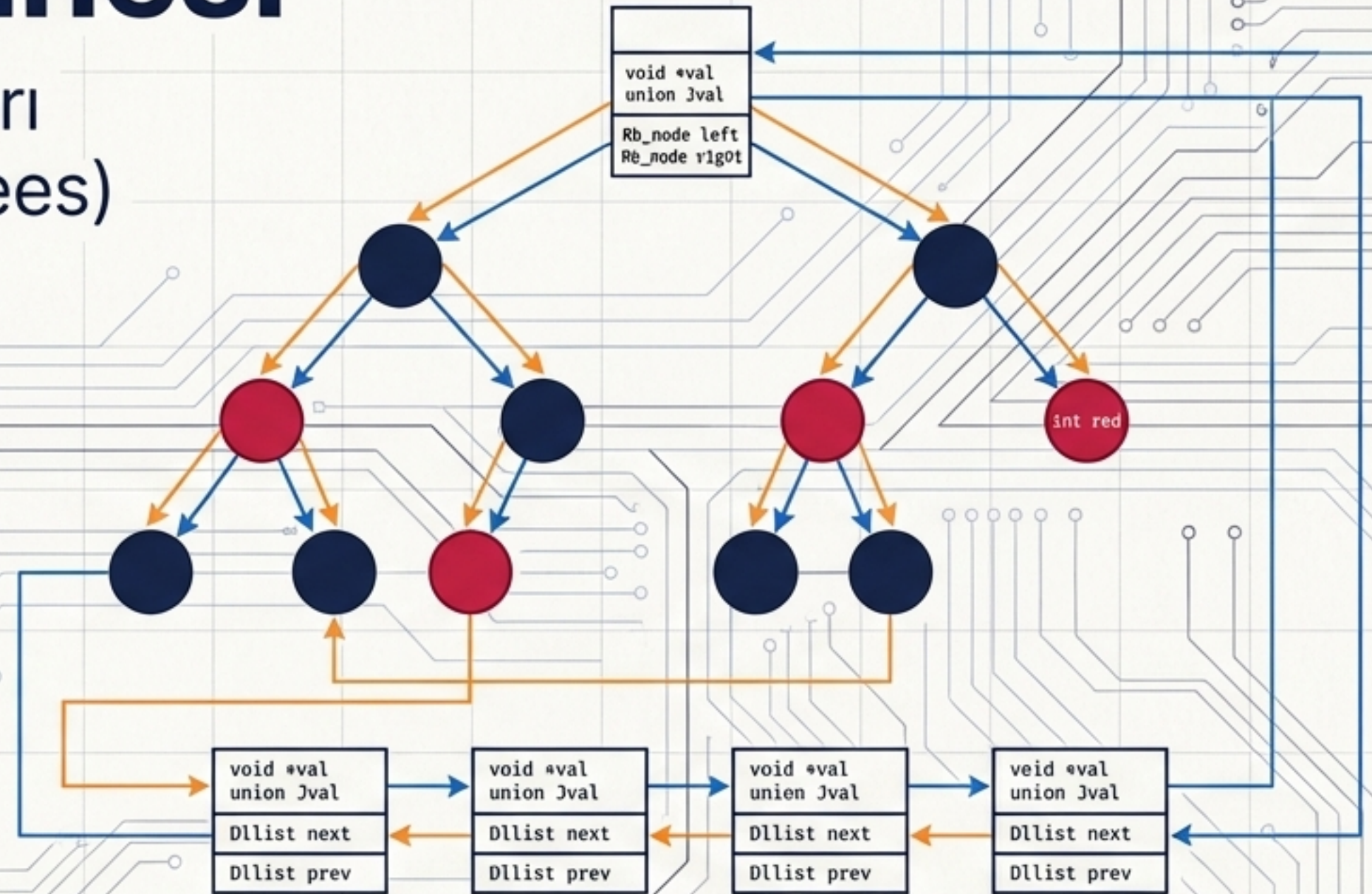


Sistem Programlama: Libfdr Kütüphanesi

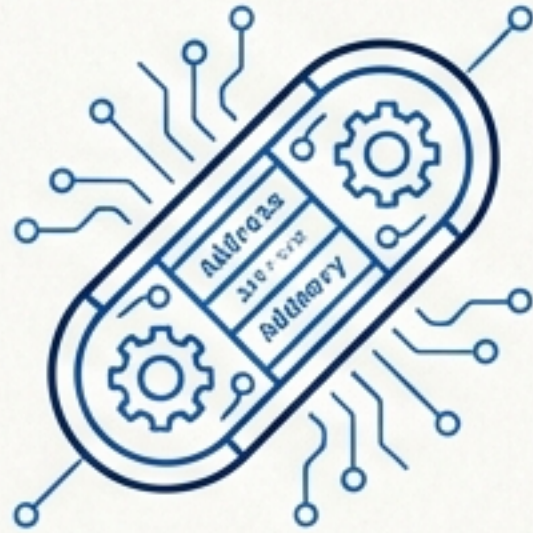
C Dilinde Jenerik Veri Yapıları
(Jvals, Dllists, Red-Black Trees)



C'nin Tip Sınırlarını Aşmak: Neden Libfdr?

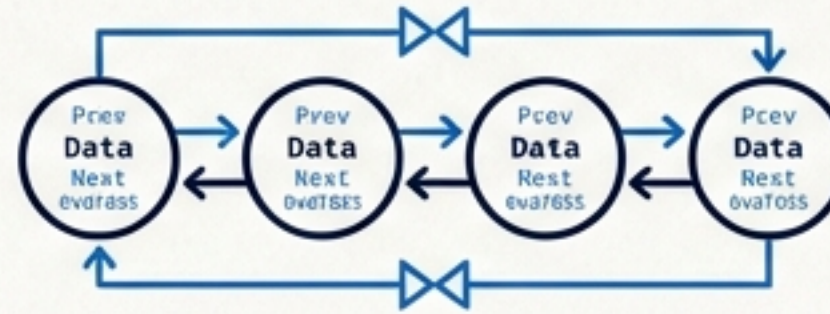
C dili dinamik ve jenerik veri yapıları için standart bir kütüphane sunmaz. Libfdr, bu yapısal engeli aşmak için üç temel mimari yapı taşı sağlar.

1. Jvals



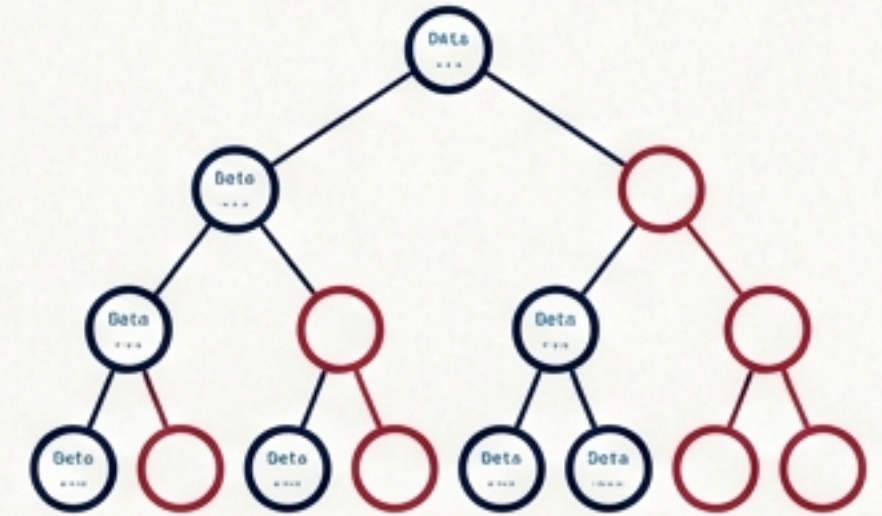
Her tipi tutabilen genel veri türü.

2. Dllists



Çift yönlü dairesel bağlı listeler.

3. RB-Trees



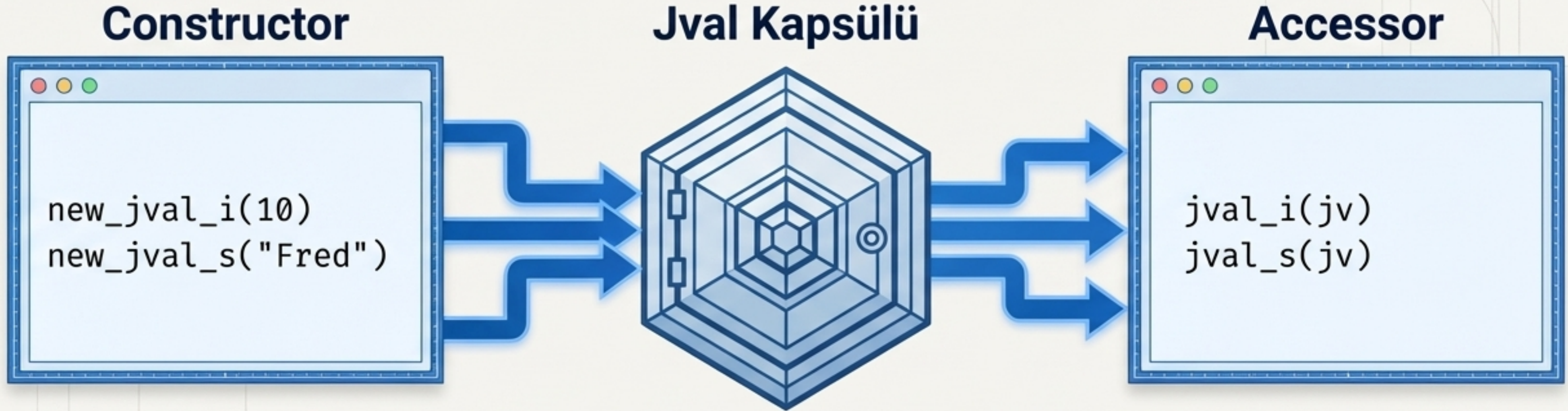
Yüksek performanslı arama/sıralama ağaçları.

Jval: C'nin Evrensel Adaptörü

Jval bir union yapısıdır. Her şeyi tutabilir, ancak aynı anda yalnızca bir yüzünü kullanabilirsiniz. Kütüphanedeki tüm esnek veri yapılarının temelini oluşturur.



Jval Yaşam Döngüsü: Kurulum ve Erişim



Uyarı: Hangi veri tipiyle oluşturulduysa, o veri tipiyle erişilmelidir. Yanlış eşleşme Garbage (çöp) değer üretir.

`jval_d(new_jval_i(10)) -> ÇIKTI: 2.13063e-314`

Jval'in Amacı Dışı Kullanımı: Okunabilirlik Katliamı

Jval sadece jenerik veri yapıları (dlist, rb-tree) için kullanılmalıdır. Temel matematiksel işlemler için native C tiplerini tercih edin.



Yanlış Kullanım

badavg.c

```
Jval total;  
Jval j;  
  
total.i += j.i;  
total.d = ((double) total.i) / n;
```

Gereksiz karmaşa,
okunabilirliği yok eder.



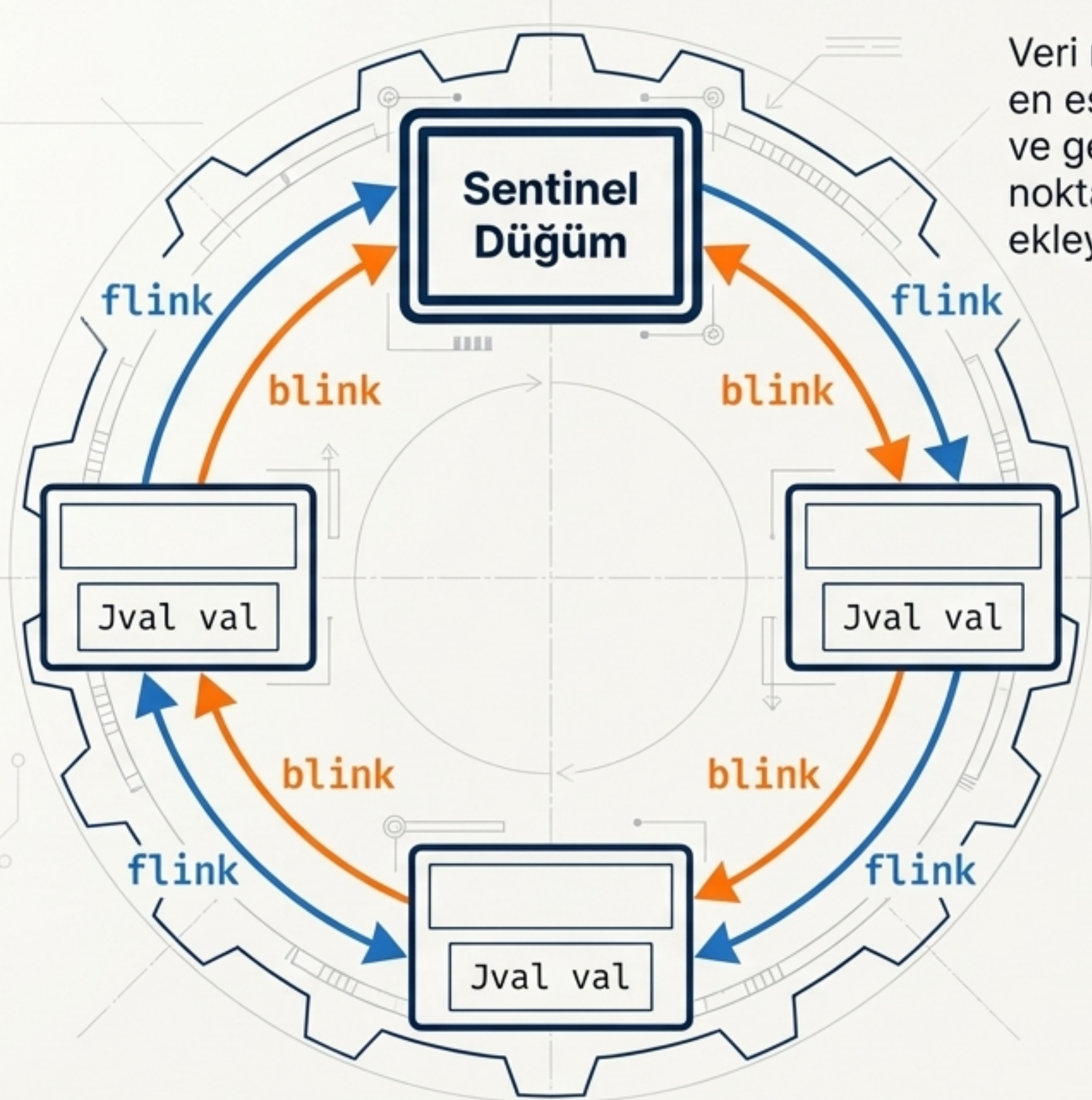
Doğru Kullanım

goodavg.c

```
double total;  
double n;  
  
total += j;  
printf("Average = %lf\n", total/n);
```

Sade, hızlı, doğrudan.

Çift Yönlü Bağlı Listeler (Dlist): Anatomik Model



Veri miktarınız belirsiz olduğunda en esnek depolama çözümü. İleri ve geri hareket edebilir, her noktadan $O(1)$ hızında eleman ekleyip silebilirsiniz.

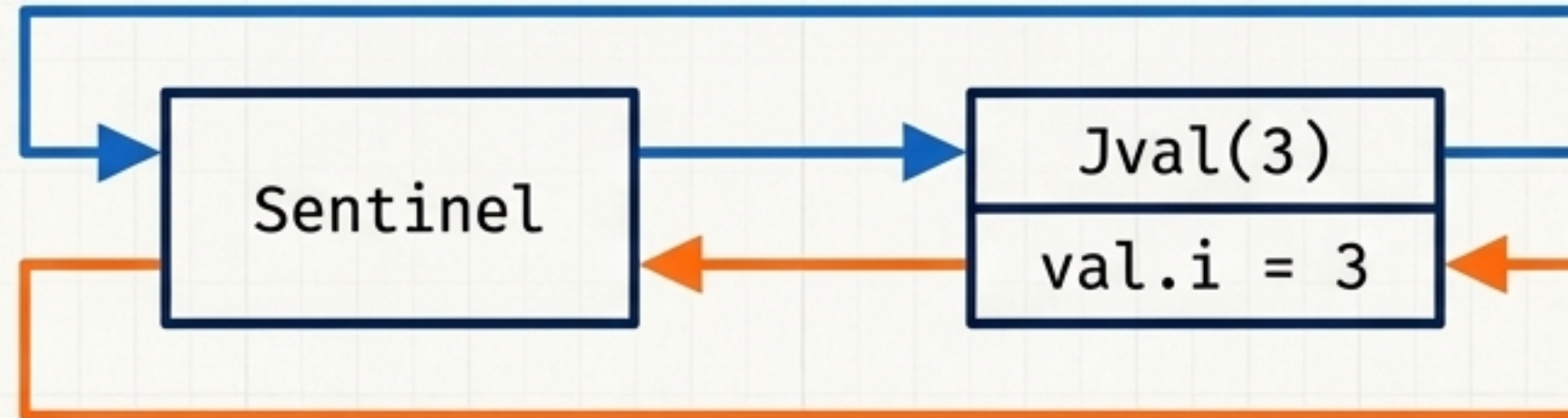
Dlist Mimarisi: Boş ve Dolu Durumlar

Sentinel düğüm her zaman mevcuttur. Liste boşken kendini işaret eder. Eleman eklendiğinde zincir kırılır ve dairesel yapı yeni düğümü içine alır.

Boş Durum (`new_dlist()`)



Dolu Durum (`dll_append(l, new_jval_i(3))`)



Dlist İşlem Komuta Merkezi

Uyarı: `dll_delete_node` işlemini asla Sentinel (başlangıç) düğümü üzerinde çağırmayın!



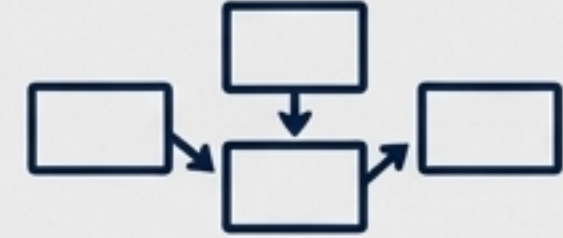
Oluştur / Yok Et

`new_dllist()`, `free_dllist()`



Uçlara Ekle

`dll_append()`, `dll_prepend()`



Araya Ekle

`dll_insert_b()`, `dll_insert_a()`



Konum

`dll_first()`, `dll_last()`,
`dll_next()`, `dll_prev()`



Veri Oku

`dll_val()`



Sil

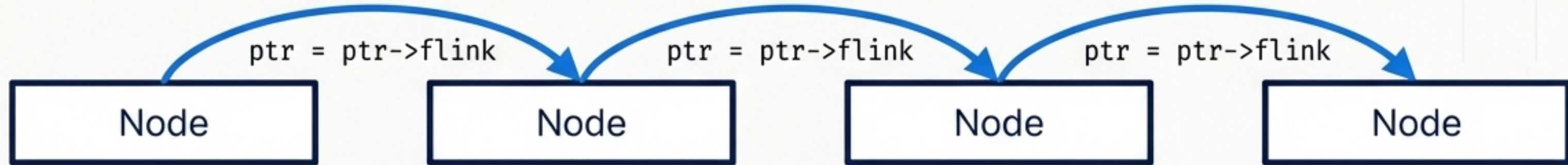
`dll_delete_node()`

Makrolarla Dlist Üzerinde Navigasyon

Karmaşık for döngüleri yerine, sentaksıı santaksı sadeleştiren gezinme makroları.

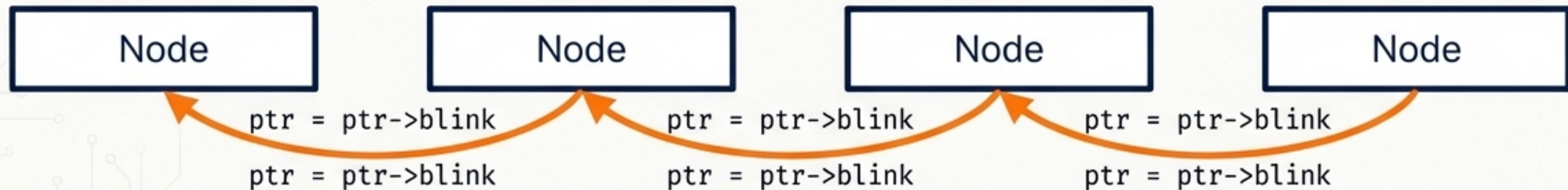
İleri Okuma

```
#define dll_traverse(ptr, list)
```



Geri Okuma

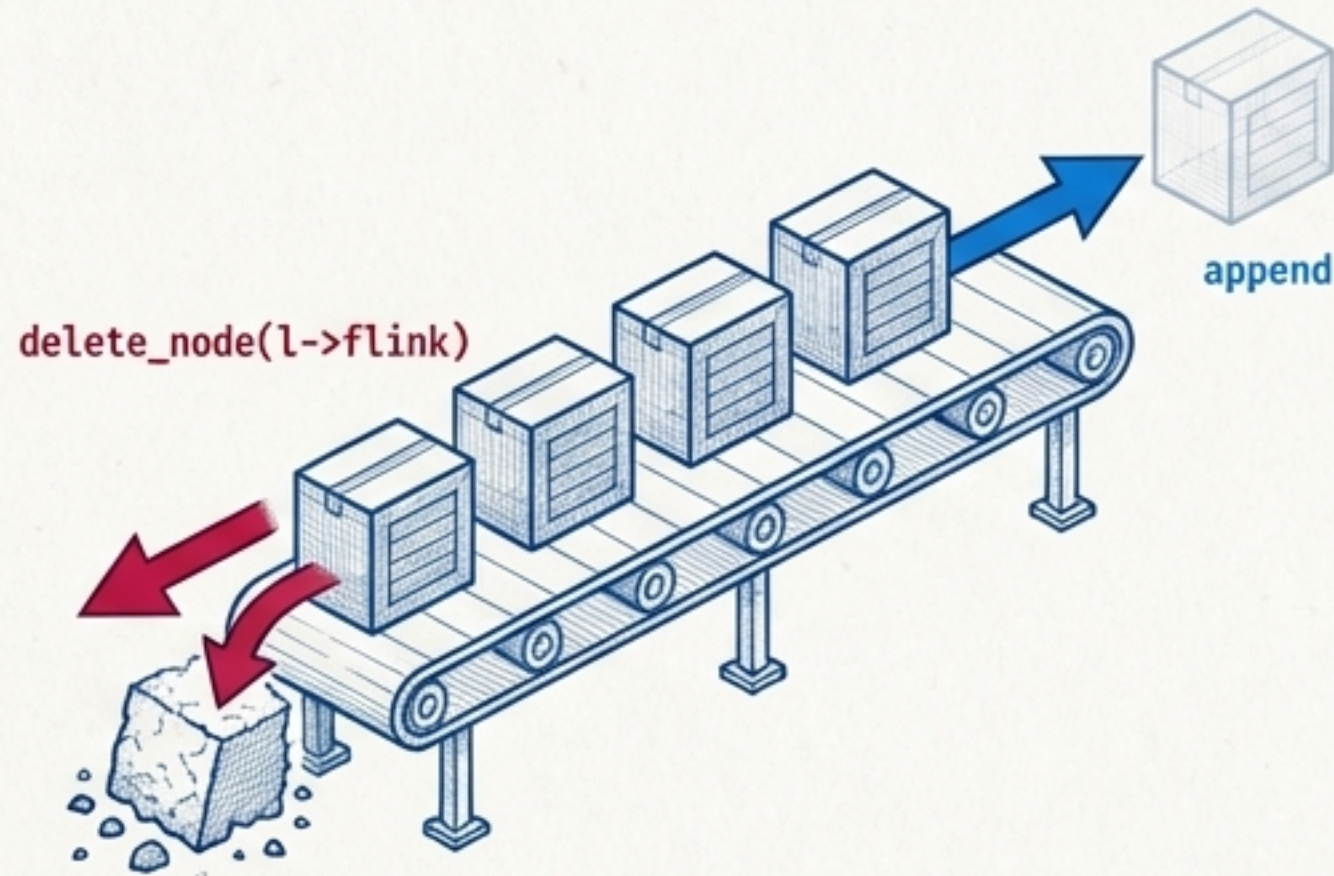
```
#define dll_rtraverse(ptr, list)
```



Dlist Uygulama Senaryoları

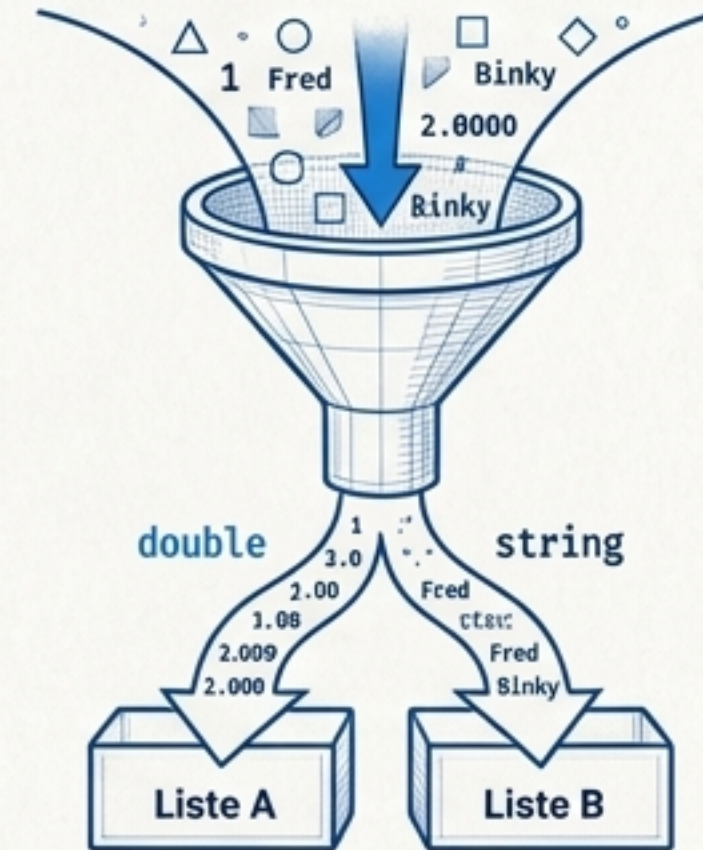
Esnek yapının gerçek dünya problemlerine çözümleri.

Senaryo 1: Son N Satırı Tutmak (Tail)



Mantık: Listeye her satırı ekle. Eleman sayısı N'i aşarsa, listedeki ilk elemanı baştan sil.

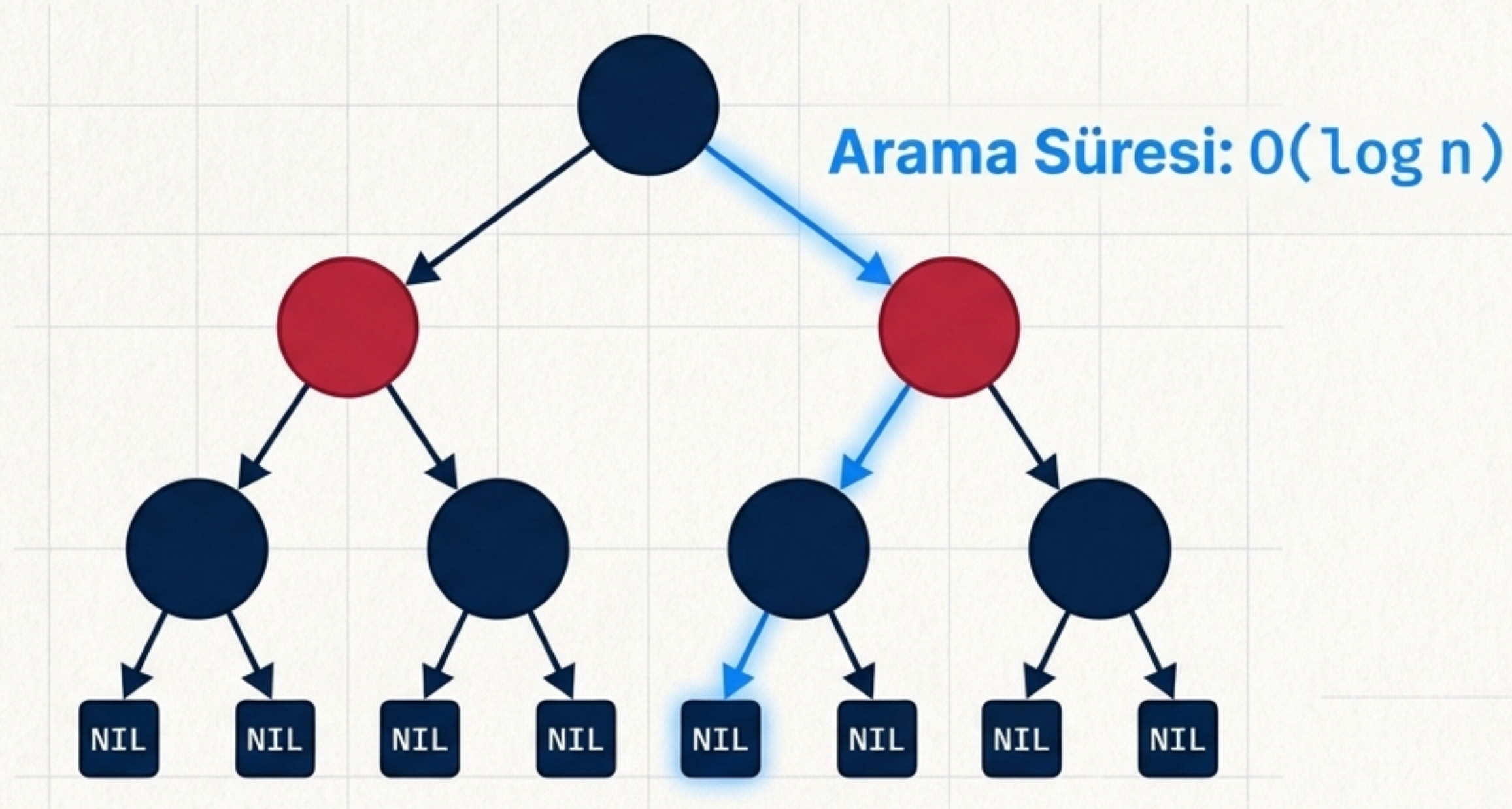
Senaryo 2: Veri Ayırıştırma (COS)



Mantık: Gelen veriyi (**Jval**) incele. Türüne göre farklı listelere dağıt ve iki sütun halinde yazdır.

Kırmızı-Siyah Ağaçlar (JRB): Hiyerarşik Hız

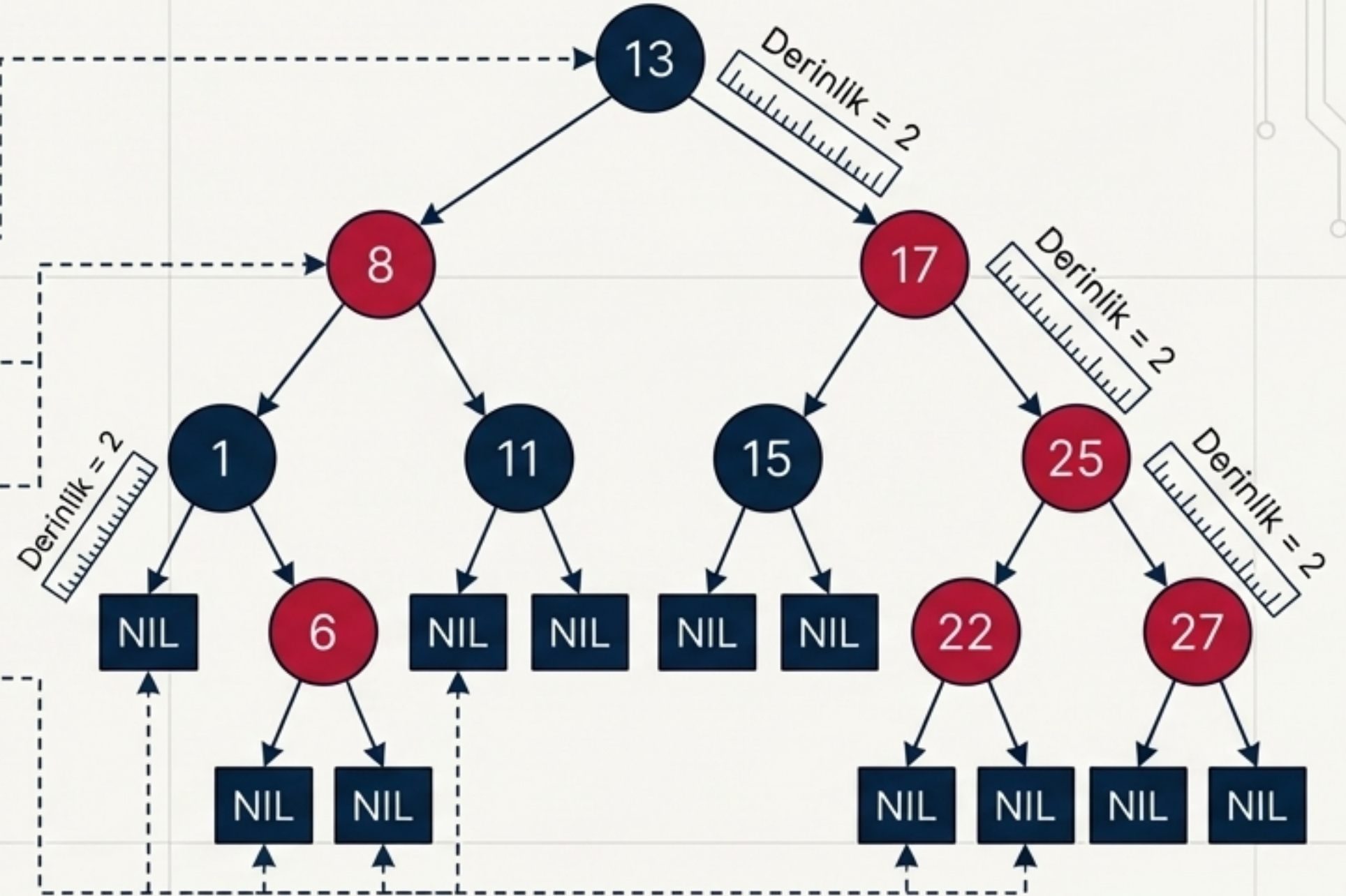
İç mimarisini manuel olarak yönetmenize gerek yok. Dev veri setlerinde arama, ekleme ve silme işlemlerinin tümü garanti edilen $O(\log(n))$ hızında gerçekleşir.



RB-Tree'nin 5 Kutsal Kuralı

Sistemin mükemmel dengede ($O(\log n)$) kalmasını sağlayan yapısal zorunluluklar.

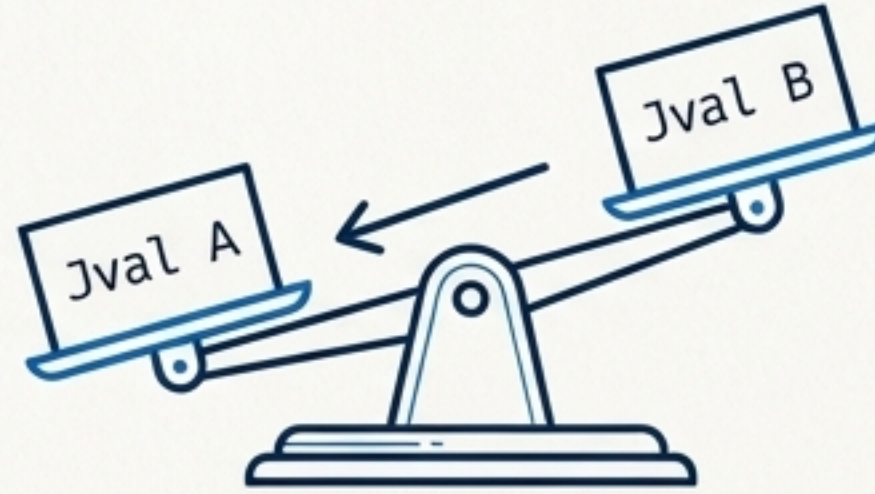
1. Her düğüm kırmızı veya siyahtır.
2. Kök siyahtır.
3. Yapraklar (NIL) siyahtır.
4. Kırmızı düğümün çocukları daima siyahtır.
5. Her yoldaki siyah derinliği eşittir.



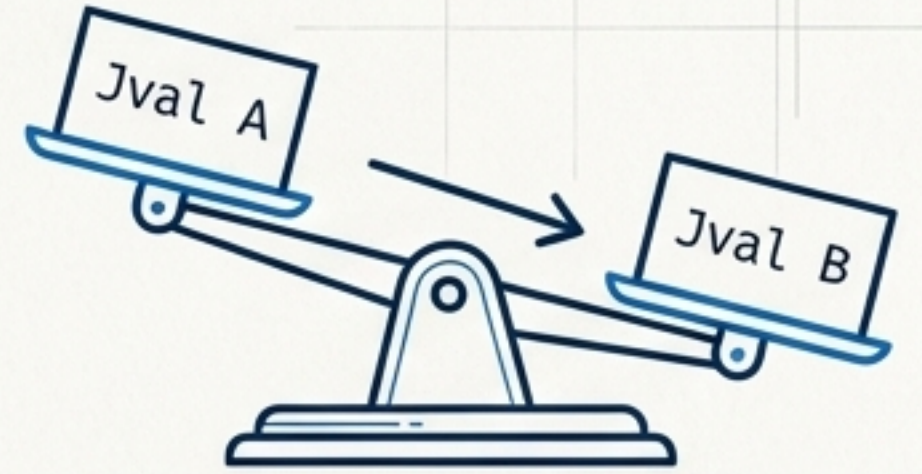
JRB'ye Veri Ekleme ve Jenerik Karşılaştırma

Sistemin her tür veriyi sıralayabilmesi için nasıl karşılaştıracağını bilmesi gerekir. **jrb_insert_gen** fonksiyonu bu işi özel bir karşılaştırma (**func**) mantığına devreder.

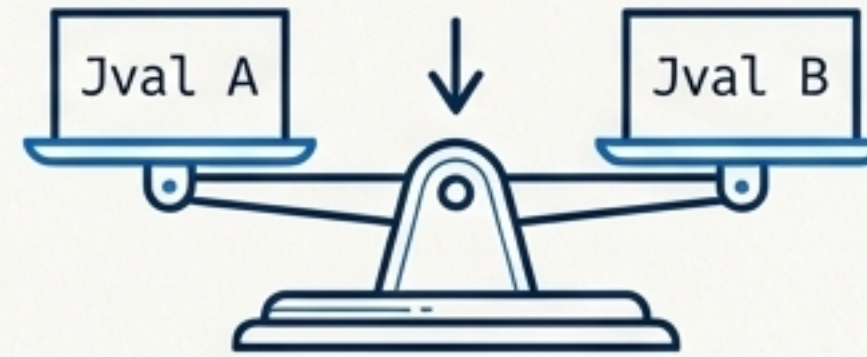
jrb_insert_str
jrb_insert_int
jrb_insert_dbl
jrb_insert_gen



Sol Ağır (Küçük): func()
Negatif Tamsayı döndürür (Sola Dallar).



Sağ Ağır (Büyük): func()
Pozitif Tamsayı döndürür (Sağa Dallar).

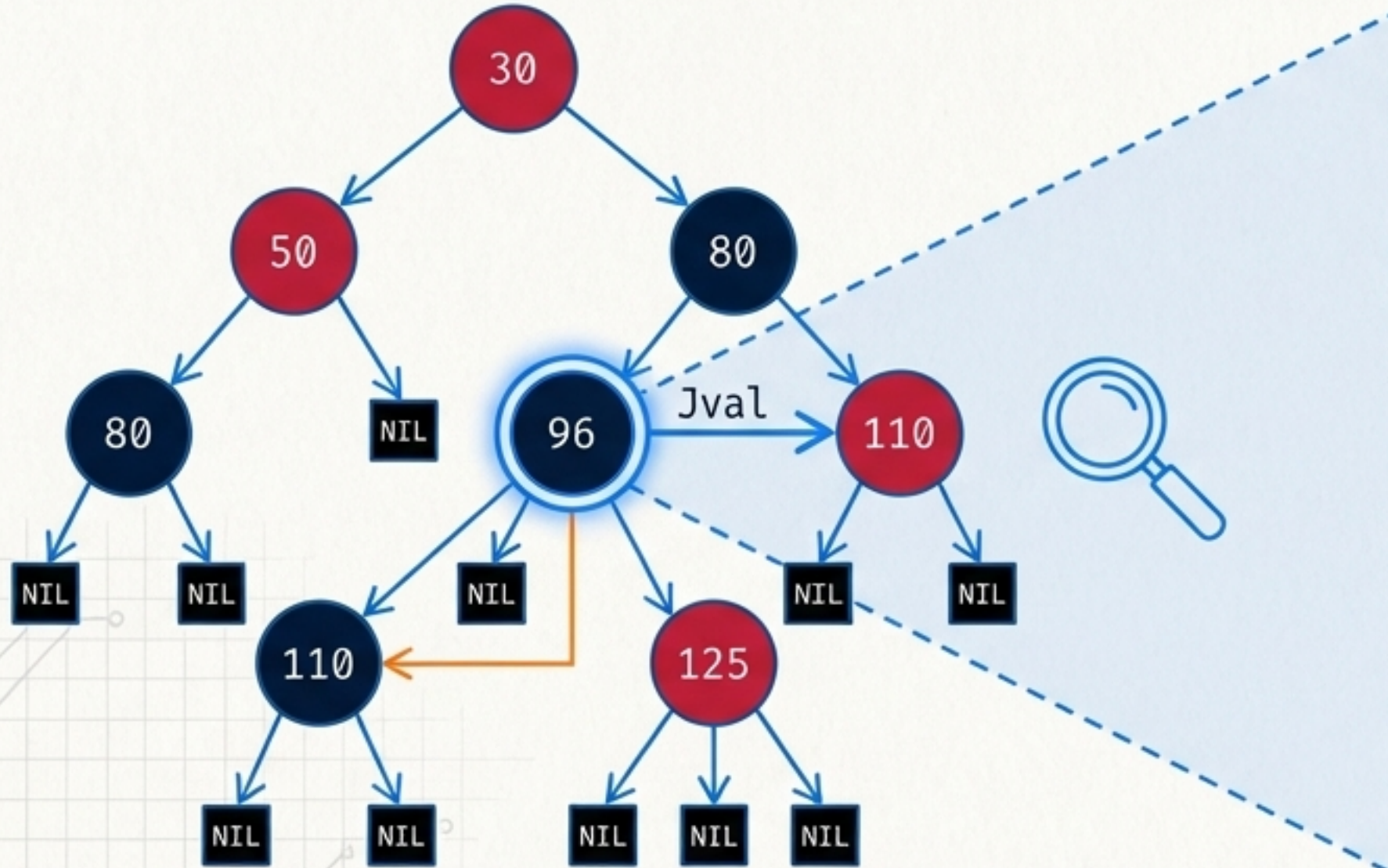


Dengede ise (Eşit): func()
Sıfır döndürür.

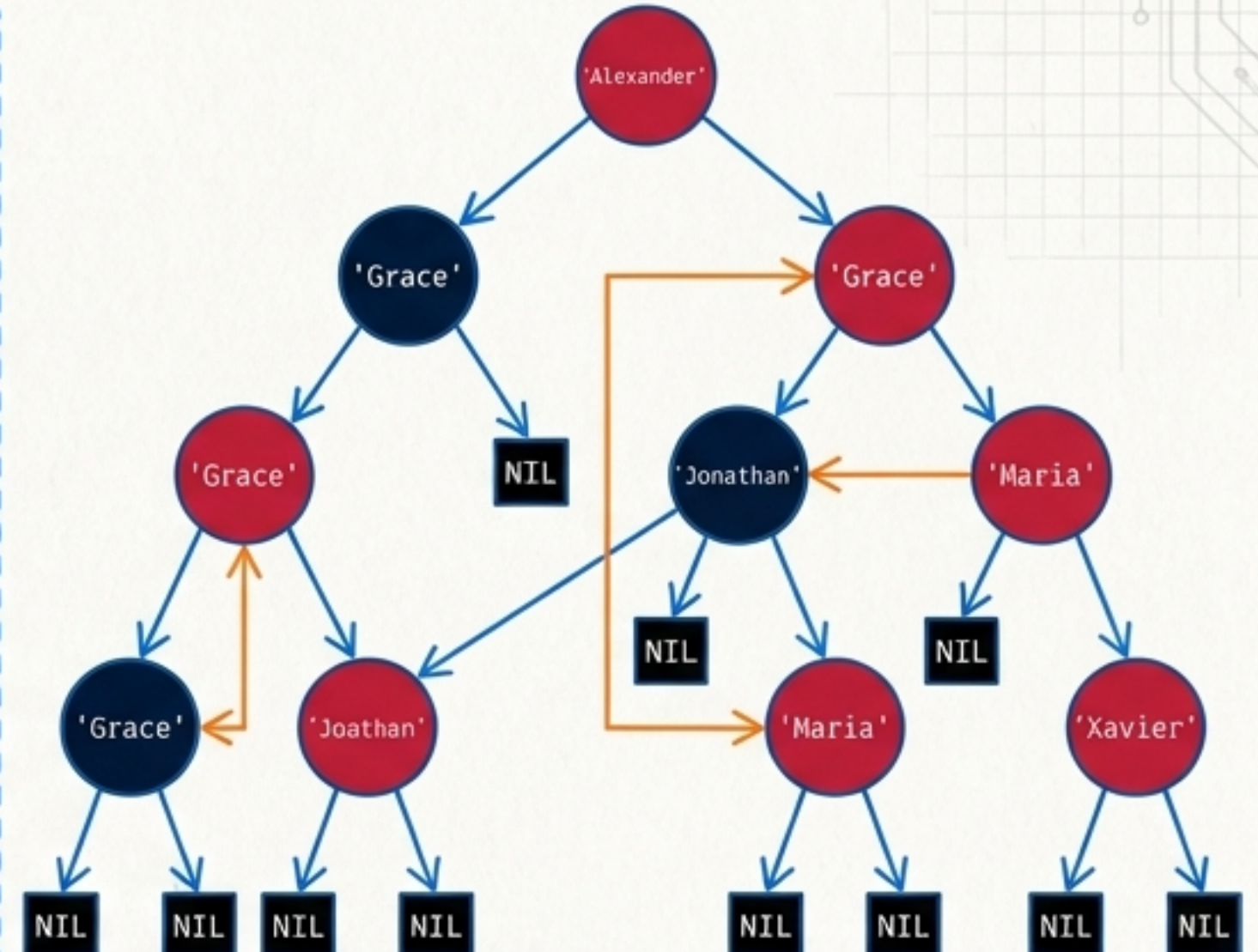
İki Seviyeli (Two-Level) Ağaç Mimarisi

Bir düğümün val (değer) alanı, başka bir Red-Black ağacını işaret edebilir!
Karmaşık çoklu-sıralama işlemleri için kusursuz çözüm.

Sayısal Değerlere Göre Sıralı (Ana Ağaç)



İsimlere Göre Sıralı (Alt Ağaç)



Sistem Mimarının Karar Matrisi

Libfdr: C dilindeki yapısal engelleri aşmanız için tasarlanmış profesyonel alet çantanız. Karşınzdaki probleme uygun yapıyı seçin.

Veri Yapısı	Boyut	Erişim	Hız Karakteristiği	İdeal Senaryo
Standart C Array	Sabit	Index ile	$O(1)$ Rastgele	Sınırları kesin olarak bilinen donanım seviyesi işlemler.
Dlist (Bağlı Liste)	Dinamik	Baştan/Sondan/Sırayla	Uçlarda $O(1)$, Arama $O(n)$	Veri akışı (stream), log kayıtları, yığın/kuyruk (stack/queue) simülasyonu.
RB-Tree (JRB)	Dinamik	Anahtar ile	Tüm işlemler $O(\log n)$	Dev veri setlerinde sürekli sıralı tutma, hızlı arama, indexleme.